

## Highlights

### **Completion-aware search with partial witness repair for dense fixed-shape facility layout**

Junhwan Ahn, Kunwoo Lee, Kyunghyun Lee, Yeoneung Kim

- Every method starts from the same verified feasible layout and keeps it.
- Partial repair reuses witness geometry after a proposed placement.
- It improves matched-time quality at fill 0.95 in two geometry families.
- A reuse-only control isolates the additional effect of partial repair.
- A small-destroy control checks whether repair size alone explains the gain.
- No advantage over full regeneration is established at the one-second cutoff.

# Completion-aware search with partial witness repair for dense fixed-shape facility layout

Junhwan Ahn<sup>b,1</sup>, Kunwoo Lee<sup>b,1</sup>, Kyunghyun Lee<sup>a,\*</sup>, Yeoneung Kim<sup>c,\*</sup>

<sup>a</sup>*Department of Computer Engineering, Gachon University, Republic of Korea*

<sup>b</sup>*Department of Applied Artificial Intelligence, Seoul National University of Science and Technology, Republic of Korea*

<sup>c</sup>*Department of Industrial Engineering, Yonsei University, Republic of Korea*

---

## Abstract

Dense fixed-shape facility layout is difficult because a legal placement can leave no completion for the remaining facilities. Suppose, however, that a verified feasible layout is already available. Then any search can retain it and verify candidates only at the end. Feasible return alone is therefore not an algorithmic advantage. We study a stricter question: under the same initial layout and the same time budget, when does maintaining a compatible completion improve solution quality? Our method keeps a completion witness and repairs only the witness facilities displaced by a proposed placement. All complete layouts are checked by a verifier that is independent of scoring and search. We compare the method with final-verification restarts, two multi-facility destroy and repair controls, fortified rollout, full completion regeneration, and witness reuse without repair. The main confirmation has 200 new instances at fill 0.95, with 32 or 64 facilities, from guillotine and nonslicing geometry families. The common best-contact initializer succeeds on 196 instances. At a 1.0 second post-initialization cutoff, partial repair improves the initial objective by 7.98% to 9.65%, compared with 1.72% to 4.44% for final-verification search. The pooled difference is 5.93 percentage points (95% paired bootstrap interval 4.99 to 6.88). It is also 6.39 points above reuse without repair and 7.02 points above spatial destroy and repair. The interval against full completion regeneration includes zero. At fill 0.90, partial repair is 1.87 points below full regeneration. Thus its advantage over

---

\*Corresponding authors: [kyunghyunlee@gachon.ac.kr](mailto:kyunghyunlee@gachon.ac.kr), [yeoneung@yonsei.ac.kr](mailto:yeoneung@yonsei.ac.kr)

<sup>1</sup>Equal contribution

the direct controls is confined to a dense, short-budget regime. We use the classical feasible-continuation argument of fortified rollout. Our contribution is the layout-specific repair mechanism and its matched-time evaluation, not a new general feasibility principle.

*Keywords:* Facility layout, Completion repair, Fortified rollout, Large neighborhood search, Feasibility verification, Anytime search

---

## 1. Introduction

Facility layout planning considers the arrangement of functional units inside a bounded plate under hard geometric constraints and soft adjacency preferences. The design space grows exponentially with the number of units. Mathematical programming and metaheuristics have therefore been widely used (Pérez-Gosende et al., 2021; Gomes de Alvarenga et al., 2000; Şahin et al., 2020; Drira et al., 2007; Singh and Sharma, 2006). The difficulty is stronger near full packing. A legal placement can block every completion of the remaining facilities. A one-step overlap mask does not detect this event. A completion heuristic can, when successful, provide an explicit feasible continuation. Classical constrained rollout already explains how a feasible continuation can be retained as a safeguard (Bertsekas, 2005, 2020). We use this invariant, but do not claim it as a new general theorem.

For an offline layout problem, the safeguard alone is not enough to establish a useful method. A complete feasible layout is available after initialization. A simple control can retain this layout, run any search, verify complete candidates at the end, and return the best verified layout. It has the same feasible-return property and can never worsen the initializer. The relevant question is therefore

Starting from the same verified feasible layout and using the same total time, when does maintaining a compatible completion produce a better layout than incumbent-preserving search with final verification?

We answer this question with completion-aware search. Its main operation is partial witness repair. If a proposed placement intersects some rectangles in the stored completion, only those rectangles are removed and repacked. All other witness rectangles remain fixed. This is a small layout-specific destroy and repair neighborhood. A failed repair rejects only that

proposal; it is not taken as evidence that the partial layout is infeasible. The current witness supplies a safe next placement, while a separate best complete layout prevents output regression. Every complete incumbent is checked by an independent verifier.

The comparison is deliberately classical. The methods share the same best-contact initial packing, candidate objective, time accounting and final verifier. Controls include final-verification construction, two spatial multi-facility destroy and repair variants, fortified rollout, reconstruction with a new full completion after each proposal, and witness reuse without repair. The principal confirmation uses 200 fresh instances at fill 0.95. Half use the original guillotine family and half use a nonslicing pinwheel family. At a post-initialization cutoff of 1.0 second, partial repair gives larger paired improvements than final verification, both spatial destroy controls and reuse without repair. Its interval against full completion regeneration includes zero. A separate fill 0.90 study shows the boundary: when complete constructions are easier, full regeneration is better at the same cutoff.

Penalty-based and learned constructors remain useful diagnostics in this paper. They explain why current legality and completion feasibility are different and show that learning is not responsible for the guarantee. They are not the main evidence for computational advantage.

#### *Contributions.*

1. **A partial repair mechanism.** We define the facilities displaced by a proposal and reuse every unaffected witness rectangle. A verified repair supplies a new compatible completion. The best full layout and the current compatible witness are kept as different objects.
2. **An evaluation from the same initial layout.** Eight methods start from the same verified packing and receive identical post-initialization time budgets. Quality is paired by instance, stochastic seeds are averaged within an instance, and initialization coverage is separated from quality on the common initialized set. The comparison identifies a useful dense, short-budget regime and also reports where the proposed method loses.
3. **Verification and correctness checks.** We state the feasible continuation invariant as an attributed specialization of fortified rollout, use an independent final verifier, and add adversarial regression tests. For the small CP-SAT study, we distinguish optimality of the scaled integer

model from the real-valued objective and transfer solver bounds with an explicit rounding allowance.

Within each experiment, all methods share one objective implementation. It agrees with the reference scenario code to 0.0 absolute error. The final feasibility verifier does not call this objective implementation.

## 2. Related work

*Facility layout.* Fixed shape unequal area layout has been studied using mathematical programming and metaheuristics, including simulated annealing (Kirkpatrick et al., 1983), tabu search (Glover, 1989), genetic and memetic algorithms, and constructive insertion rules (Gomes de Alvarenga et al., 2000; Drira et al., 2007; Singh and Sharma, 2006). Reviews (Pérez-Gosende et al., 2021; Şahin et al., 2020) organize the field by objective and by whether departments have fixed or flexible shapes. We consider the fixed shape setting on a discrete plate. The equal area limit of the problem is the quadratic assignment problem (Koopmans and Beckmann, 1957), whose library instances with proven optima (Burkard et al., 1997) provide an external evaluation in Section 7.5; robust tabu search (Taillard, 1991) is its reference heuristic and we run it there in its literature configuration.

*Fixed outline methods.* Related fixed outline floorplanning methods avoid searching absolute coordinates directly. Sequence pairs (Murata et al., 1996) and B\*-trees (Chang et al., 2000) encode relative block order and induce compact packings; fixed outline algorithms then search these encodings under an outline constraint (Adya and Markov, 2003). These representations provide controls for the transfer experiment in Section 7.5, but they do not expose the dense lattice action mask used by our constructive decoder.

*Learning for combinatorial optimization.* Neural constructive heuristics for routing and related problems (Bello et al., 2017; Kool et al., 2019), improvement policies (Wu et al., 2021) and surveys (Bengio et al., 2021; Mazyavkina et al., 2021) show that a learned scorer can replace a hand designed one inside a classical construction. A distinct line embeds learning *inside* an exact or complete solver, with learning to branch (Gasse et al., 2019) as a representative example. Our decoder is not exact, but it uses a similar separation of roles. A classical procedure enforces feasibility, and a learned or hand designed rule can rank certified actions.

*Constraint handling.* The completion invariant has direct classical antecedents. Constrained rollout retains a feasible base trajectory, and fortified rollout uses its continuation when another acceptable trajectory is unavailable (Bertsekas, 2005, 2020). Under its acceptance rule, the final cost is no worse than the base cost. Our stored witness plays the same safeguarding role. We differ in allowing marginal or learned candidate ranking and in repairing the compatible continuation geometrically. The invariant itself is not new. It is stated in Section 5 to make the implementation contract precise.

Rejecting an assignment because a future variable would lose its domain is forward checking (Haralick and Elliott, 1980). A bounded frontier gives beam search (Ow and Morton, 1988), and ordering insertions by the gap between their best options gives regret insertion (Potvin and Rousseau, 1993). More broadly, destroy and repair is the basis of large neighborhood search and adaptive large neighborhood search (Røpke and Pisinger, 2006). Partial witness repair belongs to this family. Its specific restriction is that the destroyed set is induced by overlap with a proposed rectangle, while every compatible witness rectangle is fixed and reused. We therefore compare it with an independently operating spatial destroy-and-repair search rather than attributing performance to repair in general.

*RL for layout.* Learned layout agents differ in what an action is: Kakooee and Dillenburger (2024) draw walls sequentially, Ikeda et al. (2022) deploy units constructively, Luo et al. (2025) place rooms with cooperating agents, Heinbach et al. (2023) relocate facilities on a grid, and Mirhoseini et al. (2021) place macros on a chip canvas. What these systems share is that completion feasibility is never certified: geometry enters through reward penalties or one step masks. The improvement MDP analyzed in Section 4 is the single facility, difference reward member of this family, closest to Heinbach et al. (2023); our analysis is of that reward structure, not of any particular published system.

### 3. Problem

#### 3.1. Instance and feasible set

An instance is  $\mathcal{I} = (W, H, n, \{w_i, h_i\}_{i=1}^n, A, b, \theta)$ : a  $W \times H$  integer plate,  $n$  axis aligned facilities of fixed integer size  $w_i \times h_i$ , a symmetric preference matrix  $A \in \mathbb{R}^{n \times n}$ , boundary preferences  $b$ , and objective constants  $\theta$ . A

layout assigns each facility a lower left lattice position  $p_i \in \mathbb{Z}^2$ ; write  $R_i(p_i)$  for the rectangle it occupies. The feasible set is

$$\mathcal{F} = \left\{ (p_i)_{i=1}^n : R_i(p_i) \subseteq [0, W] \times [0, H], \text{int } R_i \cap \text{int } R_j = \emptyset \ (i \neq j) \right\}. \quad (1)$$

### 3.2. Objective

On  $\mathcal{F}$  the reported objective is

$$J(g) \equiv J_\beta(g) = \beta_{\max} + w_{\text{adj}} \sum_{i < j} c_{ij}(g) + \sum_i b_i(g), \quad (2)$$

where  $c_{ij}$  rewards proximity between facilities that prefer it ( $A_{ij} < 0$ ), rewards a normalized separation between those that prefer distance ( $A_{ij} \geq 0$ ), and  $b_i$  pays a boundary bonus when facility  $i$  is flush against a wall it prefers. Distances are center to center with each rectangle’s directional radius removed, and are normalized per pair by the largest separation the plate permits, so  $c_{ij}$  is scale free. The full expressions are those of the legacy RoomLayoutRL implementation (Ahn et al., 2026) and are reproduced in Appendix A.

The constant  $\beta_{\max}$  is the no-overlap bonus used by the evaluator. It does not change rankings or raw paired differences within an instance, but it is included in every reported value and in the denominator of each normalized improvement. Outside  $\mathcal{F}$  two further terms appear, and they matter only because the improvement formulation of Section 4 operates there: an overlapping pair contributes zero adjacency, an overlapping facility forfeits its whole boundary bonus, total overlap area is penalized, and the constant  $\beta_{\max}$  is replaced by a graded overlap bonus. Our analysis treats Eq. (2) on  $\mathcal{F}$  as the optimization problem and the off-feasible penalty and graded bonus as artifacts of the earlier formulation.

### 3.3. Scenarios and verification

Two manually defined scenarios with 32 facilities are used for calibration against that legacy implementation: OFFICE ( $36 \times 24$ , fill ratio 0.54) and CLINIC ( $28 \times 16$ , fill 0.68). One vectorized implementation of Eq. (2) serves every method in the scenario and generated suite experiments. It agrees with the original scenario code to 0.0 absolute error on random, lattice aligned, heavily overlapping layouts and layouts flush against a wall, for both scenarios and both sign conventions. The annealing baseline reproduces its reported value of 3 270.0 to within 0.1, obtaining 3 269.9. Comparisons here are therefore about search, not about objective drift.

*AI assisted workflow.* Generative AI tools assisted implementation, language editing and consistency checks within the manuscript. Every numerical result in the paper was produced by the archived executable scripts; no synthetic observations replaced an experiment, and the authors ran the code, inspected the outputs and retained responsibility for every analytical and editorial decision. The declaration in the Acknowledgements names the tools used.

#### 4. Why local legality is insufficient

The earlier formulation used single-facility translations and the reward  $J(s_{t+1}) - J(s_t)$ . At discount one this telescopes to the terminal objective; below one it weights high intermediate objectives and penalizes recovered detours. It does not impose feasibility. In the reference scenario, four corrections raised the best learned objective from 2 658.5 to 3 067.2 and feasibility from 0% to 48%, while lattice annealing reached 3 414.8 and a masked constructive rule without learned parameters reached 3 380.2. The full formulation ablation is in the Online Supplement. We retain this result as motivation, not as evidence for the proposed method.

For construction, let  $\mathcal{P}_t$  be the facilities already placed. Facility  $i \notin \mathcal{P}_t$  can be placed at

$$\mathcal{L}_i(s_t) = \{p \in \mathbb{Z}^2 : 0 \leq p_x \leq W - w_i, 0 \leq p_y \leq H - h_i, S_i(p) = 0\}, \quad (3)$$

where  $S_i(p)$  is the occupied area under its proposed rectangle. An integral image tests every lattice position. The following elementary properties state exactly what this mask does and does not establish.

**Proposition 1** (Local feasibility and exact marginal). *If every step uses Eq. (3) and every facility is placed exactly once, the complete layout is contained in the plate and overlap free. Moreover, the objective increment from placing  $i$  is exactly the sum of pair terms between  $i$  and  $\mathcal{P}_t$  plus  $i$ 's boundary term.*

*Proof.* Containment is explicit. Induction over placements gives nonoverlap. Since a legal placement creates no overlap and no old facility moves, no existing pair or boundary term changes; only terms involving  $i$  are added.  $\square$

This proposition is conditional on completing all  $n$  placements. It gives no continuation guarantee. For example, on a  $5 \times 2$  plate, placing a  $1 \times 1$  rectangle at  $(1, 0)$  leaves each of two  $2 \times 2$  rectangles individually placeable,



but they cannot both be placed. The original three-rectangle instance is feasible if the small rectangle occupies the unit gap between the two large ones. Deciding whether a partial state has a completion contains rectangle packing feasibility and is NP-hard in general (Fowler et al., 1981). This is the gap addressed next; the longer penalty and masking diagnostics are retained in the reproducibility record and Online Supplement.

## 5. Witness certified construction

### 5.1. A sufficient certificate

Let  $\mathcal{V} = \{s : \text{some sequence of legal placements completes } s\}$  be the set of viable partial layouts. We bracket  $\mathcal{V}$  rather than decide it. Let  $\text{FF}(s)$  denote the outcome of first fit packing in largest first order. We process the unplaced facilities in decreasing area and place each at the first position of  $\mathcal{L}_i$  in row major order. Define the predicate

$$\mathcal{C}(s) = \left[ \text{FF}(s) \text{ places every unplaced facility} \right]. \quad (4)$$

$\mathcal{C}$  is sufficient but not necessary for viability: if it holds it exhibits an explicit completion, so  $s \in \mathcal{V}$ ; if it fails,  $s$  may still be viable. It costs  $O(nWH)$ , the same order as the mask itself.

A certificate is only useful here if it returns the completion it found. We therefore take  $\mathcal{C}$  to return a *witness*, the explicit list of placements made by the completion heuristic. A verifier checks that all facilities occur exactly once with their fixed dimensions, every coordinate is within the plate, rectangles have disjoint interiors, and the completion agrees with the committed prefix. It does not call the objective or the witness generator. Once verified, the remaining entries stay a valid completion after any one of their placements is committed.

This is the feasible-continuation invariant used by constrained and fortified rollout (Bertsekas, 2005, 2020). We record its layout specialization to define the implementation contract, not as a claim of a new general feasibility result.

**Theorem 1** (Feasible-continuation invariant). *Consider a decoder that (i) obtains and verifies a completion witness  $w_0$  for  $s_0$ , abstaining if none is available; (ii) commits a proposed placement only when its successor comes with a newly verified completion witness; and (iii) if no proposal is accepted, commits the selected facility at its position in the current witness and carries*

the remainder of that witness forward. Then the decoder commits at every step and every completed layout it returns lies in  $\mathcal{F}$ .

*Proof.* By induction on  $t$  with hypothesis “ $s_t$  has a witness  $w_t$ ”. The base case is (i). Given  $w_t$ , condition (ii) either commits a proposal and supplies a verified  $w_{t+1}$ , or condition (iii) commits the selected facility at the position recorded in  $w_t$ . In the latter case that rectangle is legal in  $s_t$  and the remaining entries  $w_t \setminus \{i\}$  complete the successor, so they form  $w_{t+1}$ . Thus a witness exists after every commitment and a commitment is always available. Every committed rectangle is legal, so Proposition 1 applies at every step and the completed layout lies in  $\mathcal{F}$ .  $\square$

Two features matter. First, the guarantee is *selective*: it says every returned layout is feasible, not that a layout is returned for every instance.

**Remark 1** (Selective guarantee against coverage). *Theorem 1 is conditioned on a verified witness existing at  $s_0$ , so the fraction of instances on which the decoder answers at all, its coverage, is exactly the success rate of the initial witness generator. After that point the stored witness, not another successful call to the incomplete generator, is the safe fallback. Coverage is therefore a property of the initial packer and verifier and has nothing to do with the policy. We report coverage and the feasibility of returned layouts separately.*

Second, the theorem *does not constrain how the decoder selects among admissible placements*. Greedy, regret, beam or a trained policy all inherit it unchanged: feasibility is delegated to the certificate, and the selection rule is free to pursue objective value inside the certified set. This separation is the one the paper is built on.

**Remark 2** (Two decoders, and only one of them is certified). *Conditions (i) to (iii) do not allow the decoder to commit the best scoring candidate when no proposal is certified. We therefore distinguish two algorithms and never blend their claims. The decoder with witness certification abstains only when no initial witness exists and otherwise uses the stored witness as its safe fallback; Theorem 1 applies to it. The heuristic with certificate guidance instead commits the highest scoring candidate when the top  $\kappa$  all fail. It emits a layout for every instance, has no guarantee, and every number we report for it is empirical. The relevant legacy rows in the Online Supplement label which decoder produced each result.*

In every timed method we also retain a separate best verified complete layout. The compatible witness supports the next commitment; the best complete layout is the output incumbent and need not agree with the current prefix. If time expires during an incomplete completion or repair, the method returns this incumbent. Thus every initialized method has the same feasible-return coverage and cannot return a value below its common initial layout.

### 5.2. Partial repair of the compatible completion

Let  $P$  be the committed prefix,  $U$  the unplaced facilities, and  $w$  a verified placement of  $U$  that completes  $P$ . Consider placing  $i \in U$  at  $p$ . After checking containment and legality against  $P$ , define the displaced set

$$D(i, p; w) = \{j \in U \setminus \{i\} : \text{int } R_j(w_j) \cap \text{int } R_i(p) \neq \emptyset\}. \quad (5)$$

We remove  $i$  from its old witness position, keep every facility in  $U \setminus (D \cup \{i\})$  fixed, and try to repack only  $D$ . The committed prefix, the proposed rectangle, and the retained witness rectangles are obstacles. In the evaluated method, a first-fit repair is attempted when  $|D| \leq 4$ ; a failed attempt rejects the proposal but says nothing about global feasibility.

**Lemma 1** (Verified witness reuse). *If the repair places every facility in  $D$  inside the plate without overlap with the fixed obstacles or with another repaired facility, then the retained and repaired rectangles form a completion after committing  $(i, p)$ . If  $D = \emptyset$ , the retained witness alone forms this completion.*

*Proof.* The retained rectangles were pairwise disjoint and avoided  $P$  in the previous witness. By definition of  $D$ , they also avoid  $R_i(p)$ . The repaired rectangles avoid all fixed rectangles and one another. Every facility in  $U \setminus \{i\}$  therefore occurs exactly once in a rectangle contained in the plate.  $\square$

The lemma is an implementation tool rather than a novelty claim. The operation is a restricted destroy-and-repair neighborhood. Its possible benefit is computational: packing  $|D|$  facilities can be cheaper than regenerating all remaining facilities. The final verifier still checks the whole completion, so the reduction concerns witness generation rather than the trusted check. The experiments log  $|D|$ , repair success, reuse, full completion calls and verifier calls to test whether this saving improves the anytime objective.

### 5.3. *Secondary certificate diagnostics*

The older full-regeneration decoder compared deterministic first fit, randomized completion and a learned predictor. Randomized attempts accepted more prefixes, but increased cost; the predictor reached validation AUC 0.902 and still admitted states that its target rollout could not complete. These experiments distinguish a rollout label from true completion feasibility and show why every accepted completion must be verified. They do not establish the main quality-time claim. Additional diagnostics and the certificate gallery are in the Online Supplement; the complete run-level record includes the learned-selector failures in the reproducibility archive.

### 5.4. *Completion-aware search with partial repair*

Algorithm 1 states M1, the principal method. Facilities are visited in decreasing area in the first reconstruction and in randomized orders afterwards. Positions are ranked by their exact marginal contribution. The position already stored for the selected facility is appended to the top  $\kappa$  candidates, so a compatible commitment is always available. The search continues until its post-initialization deadline.

---

**Algorithm 1** Completion-aware search with partial witness repair (M1).

---

```
1:  $(ok, w) \leftarrow \text{BESTCONTACT}(\emptyset)$ ; verify  $w$ 
2: if  $\neg ok$  then return NOT INITIALIZED
3:  $g_{\text{best}} \leftarrow w$ ;  $J_{\text{best}} \leftarrow J(w)$ 
4:  $B \leftarrow \text{CLOCK}() + T$  ▷ post-initialization deadline
5: while  $\text{CLOCK}() < B$  do
6:    $P \leftarrow \emptyset$  ▷ committed prefix for one reconstruction
7:   for facility  $i$  in the selected order do
8:      $Q \leftarrow$  top  $\kappa$  legal positions by exact marginal score
9:     append  $i$ 's position in  $w$  to  $Q$  if absent
10:    for  $p \in Q$  in decreasing score do
11:       $D \leftarrow D(i, p; w)$ 
12:      if  $D = \emptyset$  then
13:         $w' \leftarrow w \setminus \{i\}$  ▷ reuse unchanged remainder
14:      else if  $|D| \leq 4$  then
15:         $w' \leftarrow \text{REPAIR}(D \mid P, (i, p), w \setminus (D \cup \{i\}))$ 
16:      else
17:         $w' \leftarrow \text{FAILURE}$ 
18:      end if
19:      if  $w' \neq \text{FAILURE}$  then
20:         $g' \leftarrow P \cup \{(i, p)\} \cup w'$ 
21:         $(ok, J', t_{\text{done}}) \leftarrow \text{VERIFYANDSCORE}(g')$ 
22:        if  $t_{\text{done}} > B$  then
23:          discard  $g'$ ; return  $g_{\text{best}}$ 
24:        end if
25:        if  $ok$  then
26:          record  $g'$  as an eligible completed layout
27:          if  $J' > J_{\text{best}}$  then
28:             $g_{\text{best}} \leftarrow g'$ ;  $J_{\text{best}} \leftarrow J'$ 
29:          end if
30:          commit  $(i, p)$  to  $P$ ;  $w \leftarrow w'$ ; break
31:        end if
32:      end if
33:    end for
34:    if the deadline is reached then
35:      return  $g_{\text{best}}$ 
36:    end if
37:  end for
38:   $w \leftarrow g_{\text{best}}$  ▷ start the next reconstruction from the incumbent
39: end while
40: return  $g_{\text{best}}$ 
```

---

The witness position in  $Q$  has  $D = \emptyset$ , so the inner loop always has a verified fallback unless the deadline interrupts it. In that case the separate

complete incumbent is returned. M0 replaces the repair branch by full regeneration of all remaining facilities; like M1, it reuses the carried witness action itself. R0 accepts only nonfallback proposals with  $D = \emptyset$  and never calls repair, so it isolates witness reuse from repair. B3 evaluates the full completed objective for each successful full regeneration and selects a non-worsening continuation, which is fortified rollout. The learned selector studied later can replace marginal ranking, but it is not used in the primary comparison.

With  $r$  unplaced facilities, full first-fit completion costs  $O(rWH)$  per query. Partial generation costs  $O(|D|WH)$  when  $|D| \leq 4$ , while the independent whole-completion verifier remains  $O(nWH)$ . We therefore do not infer a speedup from the complexity expression alone. We measure calls, repair sizes and wall-clock quality. All matched-time experiments use  $\kappa = 32$ .

### 5.5. Comparison from the same initial layout

Table 1 lists the methods and controls used in the main comparison. Every method starts from the same verified best-contact layout  $w_0$ , keeps it as an incumbent, and uses the same independent verifier. B1 isolates the value of intermediate completion checks. B2 is a spatial large-neighborhood control: it removes 2 to 16 facilities near a random anchor and repacks them. B2S uses the identical implementation but limits the destroy size to 2 to 4. B3 is fortified rollout. M0, R0 and M1 have the same marginal candidate order. For every nonfallback proposal, M0 regenerates all remaining facilities, R0 accepts only  $D = 0$ , and M1 applies Eq. (5) with a repair cap of four. The guillotine and nonslicing instance families are described in Section 6.1.

Coverage is an initializer property here, not a methodwise output metric. The legacy comparison further showed that the carried witness prevents output infeasibility but does not by itself improve its objective, and that an exact marginal selector can match or exceed the learned selector. We therefore use a common best-contact initializer and training-free selectors in the primary comparison. Selected witness-frontier and selector diagnostics are reported in the Online Supplement, with complete tables in the reproducibility archive.

## 6. Experimental design

### 6.1. A generated benchmark

Repeating a solver on a fixed brief measures solver variability, not instance variability, so the two reference scenarios cannot support a statement about

Table 1: Methods in the main comparison. All start from the same initial layout and retain the best verified complete layout found.

| ID  | method                           | role                                                           |
|-----|----------------------------------|----------------------------------------------------------------|
| B0  | initial witness                  | return $w_0$ without post-initialization search                |
| B1  | final verification               | randomized construction; verify only complete candidates       |
| B2  | spatial destroy and repair       | independent multi-facility improvement search                  |
| B2S | small spatial destroy and repair | B2 with destroy size restricted to 2 to 4                      |
| B3  | fortified rollout                | rank by completed objective and enforce non-worsening          |
| M0  | full regeneration                | marginal order; regenerate nonfallback proposals               |
| R0  | reuse only                       | marginal order; accept nonfallback proposals only when $D = 0$ |
| M1  | partial repair                   | marginal order with bounded witness reuse and repair           |

how a method behaves across problems. We generate a held out suite and treat the *instance* as the unit of analysis.

*Feasibility by construction.* Sampling facility sizes independently produces instances that cannot be packed at high density, and a suite containing impossible instances measures nothing. The original suite is built from a guillotine partition, recursively slicing the plate into  $n$  disjoint integer rectangles and shrinking each inside its own slot.

The second family starts from a tiled five-rectangle pinwheel with no full horizontal or vertical guillotine cut. A source rectangle is selected and split at an integer coordinate. The split is accepted only if the resulting tiling still has no full guillotine cut. This procedure is repeated until the tiling has  $n$  source slots. The rectangles are then reduced by a common scale factor, rounded down to integer dimensions, and enlarged one cell at a time without leaving their respective slots until the target fill is reached. The nonslicing condition is checked on the complete source tiling before this reduction. After the reduction, gaps are present and the same full-cut test for a tiling is no longer applicable. We therefore do not claim that the reduced layout is nonslicing under a definition that allows dead space.

For both families, the lower-left corners of the source slots give a feasible witness. Its boundary and overlap conditions are independently verified before any solver is run, but the coordinates are withheld from every method. The implementation is in `rl_layout/v2/bench.py`, the structural checks are in `rl_layout/v2/test_bench_families.py`, and the experimental entry point is `rl_layout/v2/run_completion_comparison.py` in the accompanying archive.

*Design.* The suite crosses four facility counts,  $n \in \{8, 16, 32, 64\}$ , with three fill ratios,  $f \in \{0.40, 0.60, 0.75\}$ , at 20 instances per cell: 240 instances. Five extensions follow the same construction from disjoint seeds. Section 7.5 adds  $f \in \{0.80, 0.85, 0.90, 0.95\}$  at  $n = 32$  (80) and  $f \in \{0.85, 0.90, 0.95\}$  at  $n = 64$  (60); Section 7.5 adds  $n \in \{96, 128, 192, 256\}$  at  $f = 0.85$  and  $f = 0.95$ , with plates enlarged in proportion so facilities do not shrink to the minimum size (180). The certified sweep adds the missing  $n = 64, f = 0.80$  cell (20). Finally, the frozen decoders are run on a confirmatory family of 100 new instances in each  $n \in \{32, 64\}$  by  $f \in \{0.90, 0.95\}$  cell (400), and a small plate family at  $n = 8$  (60), sized so that constraint programming can prove optima, supports the gap study of Section 7.4. These legacy diagnostic suites contain 1 040 distinct held out instances. The main comparison adds 200 dense confirmation instances and 80 boundary-study instances under separate tags. Every source packing is verified. Achieved fill ratios match their targets to within 0.001. Plate aspect ratio, the type inventory, the preference matrix, boundary preferences and the objective constants are redrawn per instance; facility sizes come from the partition and types are drawn independently of geometry.

*Splits.* Train, validation and test draw from disjoint generator seed ranges, so an instance evaluated at test time cannot have been optimized in training. Instance identity is (family, split,  $n$ ,  $f$ , index) and is reproducible from the seed. The earlier policy confirmation is tagged `dense100_v3`. The new comparison families use separate v4 tags; they are disjoint from training, validation and all earlier test cells.

## 6.2. Primary comparison protocol

The dense study crosses two geometry families and  $n \in \{32, 64\}$  at fill 0.95, with 50 attempted instances per cell. The boundary check uses the same four cells at fill 0.90, with 20 attempts per cell. The common best-contact



initializer succeeds on 196 of 200 dense instances and 79 of 80 boundary instances. Quality comparisons use these fixed common initialized sets.

All methods are run once to a 1.0 second post-initialization cutoff. Values at 0.1 and 0.3 seconds are read from the same anytime trace. The dense study uses two evaluation seeds per stochastic method and the boundary study uses three. Evaluation seeds are averaged within an instance before any paired comparison. The instance is the inference unit, and intervals use 5,000 paired instance bootstrap samples. A difference of one percent of  $J(w_0)$  is the fixed practical-effect threshold.

The tuned parameters are  $\kappa = 32$ , M1 repair cap 4, and B2 destroy cap 16. B2 draws the destroy size uniformly from 2 through the cap, selects the nearest facility centers to a random anchor, optionally shuffles their decreasing-area order, and samples from a restricted list of the four strongest among 12 candidate positions. Only a complete independently verified improvement can replace the incumbent. On 20 disjoint validation instances at  $n = 64$ , fill 0.95, and 0.3 seconds, B2 caps 4, 8, 12 and 16 improved the initial objective by 1.06%, 1.00%, 1.15% and 1.34%, respectively; cap 16 was then fixed. B2S repeats the test implementation with cap 4 to test small destroy sizes directly.

### 6.3. Methods

The primary methods B0 to B3, B2S, M0, R0 and M1 are defined in Table 1. They are classical CPU implementations and share the initial layout, objective, time limit and verifier. The following methods belong to the earlier diagnostic experiments.

- **Classical search:** shelf packing; lattice constrained simulated annealing at 8 192 and 45 000 proposals; tabu search (Glover, 1989) over single facility moves; and a lattice genetic algorithm with uniform crossover and elitism. Annealing and tabu use periodic greedy polish and a common budget of 45 000 objective calls. Constraining annealing to the lattice is itself worth 4.4% on OFFICE and raises CLINIC feasibility from 20% to 100%; we report against this stronger version. Tabu’s tenure and start and the GA’s population and memetic setting are chosen on validation like every other hyperparameter: tabu keeps the constructive start (steepest single facility moves cannot repair a random dense start), and the memetic variant was rejected for exceeding the evaluation budget tenfold without improving on its compliant counterpart.

- **Masked constructive methods without training:** greedy with largest first order; greedy with joint choice of facility and position; regret  $k$  insertion for  $k \in \{2, 4, 8\}$ ; beam search over insertion decisions at widths 1 to 64; and beam search with the order fixed to largest first.
- **Methods without training and with certificate guidance:** largest first and joint greedy with the certificate filter of Section 5 and an argmax fallback. These are empirical heuristics, not instances of Theorem 1.
- **Learned:** a constructive policy trained with PPO (Schulman et al., 2017); its heuristic variant with certificate guidance; and its legacy full-regeneration variant with a carried witness. This last method uses the invariant of Theorem 1, but not the partial repair of Algorithm 1. Architecture and hyperparameters are in Appendix B.

Beam width and regret  $k$  are selected on the *validation* split for the objective comparison of Section 7.4, and the test split is evaluated once with them fixed. The density cells of Section 7.5 report the whole sweep instead, because the finding there is the pattern of collapse across widths. On the two reference scenarios, single instances with no validation counterpart, the best setting is taken on the scenario itself; that favors the baselines without training, and we flag it there rather than reporting a  $p$  value.

#### 6.4. Which policy covers which instances

The encoder concatenates pair features of length  $n$ , so one checkpoint serves one facility count (Section 9). No single policy spans the suite, and Table 2 states exactly which instances a learned method was run on and which are comparisons of methods without training. Aggregate instance counts elsewhere in the paper refer to held out *test* instances; training and validation instances are drawn from disjoint seed ranges and are never evaluated.

#### 6.5. Cost accounting

One “evaluation” does not mean the same work for every method, so we separate four units: full objective evaluations,  $O(n^2)$  each; candidate position scores, one exact marginal for one facility at one position; network forward passes, one of which scores all  $O(WH)$  positions; and wall clock, with hardware named. Learned inference uses a GPU and classical solvers vectorized CPU code; the comparisons our conclusions rest on are between constructive

Table 2: Datasets, and the policies evaluated on them. Training budgets and wall clock differ by policy family and are listed per checkpoint in Table B.5 of Appendix B; training cost is per policy, not amortized across rows.

| dataset                       | $n$       | inst. | policies           | trained on              |
|-------------------------------|-----------|-------|--------------------|-------------------------|
| generated, main               | 8, 16     | 120   | none               | not used                |
|                               | 32        | 60    | 3 seeds, env order | train, $f \leq 0.75$    |
|                               | 64        | 60    | see density rows   | not used                |
| generated, density            | 32        | 80    | 2 seeds, own order | train, $f=0.75$ to 0.90 |
|                               | 64        | 60    | 2 seeds, own order | train, $f=0.75$ to 0.90 |
| generated, certified only     | 64        | 20    | 2 seeds, own order | frozen; $f=0.80$        |
| generated, dense confirm      | 32, 64    | 400   | 2 seeds, own order | frozen checkpoints      |
| same initial layout, dense    | 32, 64    | 200   | none               | not used                |
| same initial layout, boundary | 32, 64    | 80    | none               | not used                |
| generated, large              | 96 to 256 | 180   | none (fixed $n$ )  | not used                |
| generated, exact              | 8         | 60    | none               | not used                |
| reference scenarios           | 32        | 2     | 1 per scenario     | that scenario           |
| MCNC / GSRC                   | 33 to 300 | 5     | none (other obj.)  | not used                |
| QAPLIB                        | 12 to 30  | 32    | none (other obj.)  | not used                |

methods scoring the same position grid, and the missing CPU only inference figure is flagged in Section 9. The primary comparison does not mix CPU and GPU methods. It records wall time from one continuous anytime run, including scoring, repair and verification. A candidate is admitted only if its independent verification and objective evaluation both finish by the cutoff. Work already in flight may finish later, but that candidate is discarded; the largest function-return overrun was 9.8 ms and the latest admitted event was at 0.999998 s. Measured initialization time is reported separately and charged equally to every method. The v5 comparison used one AMD Ryzen 7 9700X CPU process on Windows 10, with Python 3.7.16 and NumPy 1.21.5; the exact metadata are stored with the result file. The CP-SAT audit used OR-Tools 9.15 in the separate exact environment.

### 6.6. Statistics

For the main comparison, quality is defined on the fixed common initialized set. Stochastic evaluation seeds are averaged within each instance, then paired differences are bootstrapped over instances. We report raw and initial-objective-normalized differences, win/tie/loss counts, and the fixed one-percent practical threshold. Every shorter budget is a prefix of the longest run. The primary dense contrast is M1 minus B1 at 1.0 seconds;

contrasts with R0, B2S and M0 test reuse, repair scale and regeneration, respectively.

For the earlier diagnostic experiments, the following protocol is retained. Every method is attempted on every instance in the cells assigned to it. Objective comparisons are paired only on instances for which both methods return feasible layouts. We normalize by the best *feasible* objective achieved on that instance, because raw values differ by an order of magnitude between cells with 8 and 64 facilities. We report the mean paired gap, a 95% bootstrap confidence interval over instances, a Wilcoxon signed rank test on paired differences with Holm correction across the family, and the number of instances on which each method wins. For binary reliability we report exact binomial confidence intervals. In the prespecified dense confirmation, guided and unfiltered decoders are paired instance by instance using the exact two sided McNemar test, with Holm correction across all twelve selector and cell comparisons. Coverage and feasibility conditional on return are analyzed separately for the selective decoder.

## 7. Results

### 7.1. Dense results from the same initial layout

At the 1.0 second cutoff, M1 exceeds B1 by 5.93 percentage points on the 196 initialized dense instances (95% paired bootstrap interval 4.99 to 6.88; win/tie/loss 156/0/40). The raw paired mean is 224.5 objective units (172.2 to 276.5), and 152 instances meet the fixed one-percent practical threshold. The cellwise M1 minus B1 differences are 7.79, 4.33, 7.86 and 3.88 points in Table 3 order; all four intervals exclude zero.

Comparisons with the direct controls give the following results. Pooled M1 minus R0 is 6.39 points (5.89 to 6.92; 195/0/1), so reuse without nonempty repair does not explain the gain. M1 minus B2 is 7.02 points (6.54 to 7.52; 196/0/0), and M1 minus B2S is 6.33 points (5.84 to 6.82; 195/0/1). Restricting B2 to the same small destroy scale therefore does not close the gap.

The comparison with full regeneration depends on the cutoff. The pooled M1 minus M0 differences are 1.18 points at 0.1 seconds (0.58 to 1.75), 1.05 points at 0.3 seconds (0.46 to 1.67), and 0.25 points at 1.0 second (−0.42 to 0.92). At 0.1 seconds, the pooled difference is mainly due to the two  $n = 32$  cells; the M1 point estimate is lower than M0 by 0.31 and 0.33 points in the two  $n = 64$  cells. Thus, the early difference is not uniform

Table 3: Dense study at fill 0.95 and a 1.0 second post-initialization cutoff. Method columns are mean  $100[J_m - J(w_0)]/J(w_0)$  on the common initialized set. The last column is the paired M1 minus B1 difference with a pointwise 95% bootstrap interval.

| geometry   | $n$ | init. | cov. | B1   | B2   | B2S  | B3          | M0           | R0   | M1          | M1-B1 |             |
|------------|-----|-------|------|------|------|------|-------------|--------------|------|-------------|-------|-------------|
| guillotine | 32  | 49/50 | .98  | 1.72 | 2.40 | 3.51 | <b>9.63</b> | 9.04         | 2.43 | 9.51        | 7.79  | [6.04,9.44] |
| guillotine | 64  | 50/50 | 1.00 | 4.44 | 1.41 | 1.62 | 3.69        | 8.00         | 2.31 | <b>8.76</b> | 4.33  | [2.25,6.37] |
| nonslicing | 32  | 47/50 | .94  | 1.79 | 2.64 | 3.71 | 9.53        | <b>10.15</b> | 2.86 | 9.65        | 7.86  | [6.18,9.53] |
| nonslicing | 64  | 50/50 | 1.00 | 4.10 | 1.38 | 1.78 | 3.60        | 7.78         | 2.70 | <b>7.98</b> | 3.88  | [2.00,5.81] |

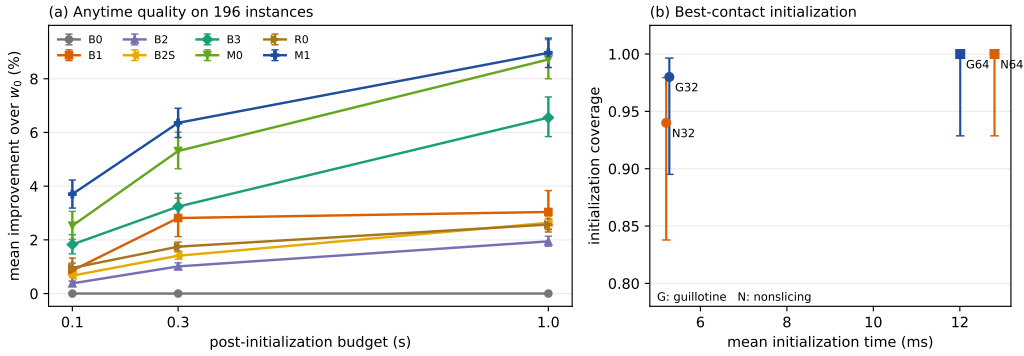


Figure 1: Quality time and initialization views for the dense study. Panel (a) pools the 196 initialized instances after averaging evaluation seeds; bars are pointwise 95% instance-bootstrap intervals. The 0.1 and 0.3 second points are prefixes of runs that continue to 1.0 second. Panel (b) reports coverage and Wilson intervals over 50 attempted instances per cell. G and N denote guillotine and nonslicing geometry.

across sizes. At one second, the interval establishes neither a difference nor equivalence between partial repair and full regeneration. The early results are consistent with the intended role of partial repair, since it repacks only the displaced rectangles. They do not support a general claim about longer runs or deployment settings.

## 7.2. Mechanism and timing checks

The mean proposed displaced-set size ranges from 1.95 to 2.72 across the dense cells. Only 2.19% to 3.91% of attempted nonempty repairs of size at most four succeed. Still, the successful repair branch produces 6,215 incumbent updates and 92% of M1’s logged objective gain, compared with 1,380 updates from unchanged reuse. These are descriptive event counts, not

independent samples. They agree with the paired R0 contrast: the nonempty repair branch, rather than reuse alone, carries the improvement.

B2’s mean destroy size is 8.88 to 9.06, while B2S’s is 2.99 in every cell. Thus the B2S comparison directly checks the concern that B2 loses only because its neighborhoods are larger. M1 processes 55.0 to 80.8 verified non-fallback completions per run, and M0 processes 35.7 to 59.6. No intermediate or final verification failure occurs.

The independent result auditor regenerates every instance, verifies all 8,820 stored budget layouts, recomputes their objectives, and checks every budget row against its eligible trace prefix. The largest objective and trace errors are both zero. In-flight work produces 194 candidates that finish after the cutoff; all are discarded. No admitted event is later than 0.999998 seconds, although a nonpreemptive call can return up to 9.8 ms after the cutoff.

### 7.3. Boundary at fill 0.90

The boundary check is based on 80 attempted instances, of which 79 share a verified initial layout. At 1.0 second, pooled M1 minus B1 is 0.23 points (−0.75 to 1.22); this interval does not establish a difference. M1 minus M0 is −1.87 points (−2.60 to −1.17). In the four cells, M1 improves the initial objective by 14.60%, 13.44%, 13.84% and 13.07%, while M0 gives 16.17%, 15.48%, 14.90% and 15.85%. Partial repair is therefore useful relative to the direct reuse and spatial controls at fill 0.95, but it is not a general replacement for full regeneration when the plate is less tight.

### 7.4. Secondary calibration and the small-instance audit

On the two hand-authored scenarios, masked constructive search supplies most of the improvement over translation-based search, but learning adds only 0.9% over the best training-free procedure in the same action set. On 60 held-out moderate-density instances, beam search without training has the best paired objective and solves 57, compared with 60 for the policies. These results keep learning out of the primary contribution; supporting tables and complete run-level tests are reported in the Online Supplement and archive, respectively.

*Small-instance objective audit.* Paired differences say which method is better, not whether any of them is good. To measure that we freeze a dedicated small plate family from the same generator ( $n = 8$ , fills 0.40 to 0.75, 20 instances

Table 4: Small-instance calibration at  $n = 8$ . The first three numeric columns are real-objective shortfalls from the returned CP-SAT solution on the 45 quantized models proved optimal. The last column uses the transferred real upper bound  $U$  on all 60 instances. Objectives and bounds are positive here, so  $(U - J)/U$  upper-bounds the relative gap to the unknown real optimum. Regret 4 is infeasible on one instance and is excluded rather than imputed.

| method                    | mean         | median       | max   | certified upper bound |
|---------------------------|--------------|--------------|-------|-----------------------|
| beam width 32             | <b>0.40%</b> | <b>0.19%</b> | 2.66% | $\leq 0.85\%$         |
| SA 45 000, lattice        | 1.48%        | 0.85%        | 9.79% | $\leq 1.74\%$         |
| regret 4 insertion        | 1.94%        | 1.41%        | 6.05% | $\leq 2.35\%$         |
| tabu 45 000, greedy start | 2.62%        | 1.96%        | 7.31% | $\leq 2.88\%$         |
| largest first greedy      | 2.77%        | 2.13%        | 7.31% | $\leq 3.00\%$         |

each, mean facility area 6 to 10 so plates stay at 66 to 195 cells). The CP-SAT model (Google OR-Tools, 2026) has integer lower-left coordinates  $X_i \in [0, W - w_i]$  and  $Y_i \in [0, H - h_i]$ , interval variables of fixed size, and a native NOOVERLAP2D constraint. For every nonzero pair, an allowed-assignment table has columns  $(u, v, q)$ , where  $u = X_j - X_i$ ,  $v = Y_j - Y_i$ , and  $q = \text{round}(10^4 w_{\text{adj}} c_{ij})$ . Boundary coefficients are rounded at the same scale and attached to reified wall-contact variables. The no-overlap bonus is constant on  $\mathcal{F}$  and is added without rounding.

Let  $a_\ell(g)$  denote an unrounded active pair or wall term, let  $M$  be the number of such terms and let  $s = 10^4$ . For every feasible layout,

$$\hat{J}(g) = \beta_{\max} + \frac{1}{s} \sum_{\ell=1}^M \text{round}(s a_\ell(g)), \quad |J(g) - \hat{J}(g)| \leq \delta := \frac{M}{2s}. \quad (6)$$

Thus a CP-SAT upper bound  $\hat{U}$  transfers to the real objective as  $U = \hat{U} + \delta$ , and an optimizer of  $\hat{J}$  is within  $2\delta$  of the real optimum. CP-SAT proves 45 of 60 *quantized* models optimal within 600 seconds and returns bounds for the other 15. The audit independently checks all returned layouts and reevaluates their real objective. The largest  $\delta$  is 0.00215 and the largest observed  $|J - \hat{J}|$  is 0.000322. We therefore make no claim of literal exact optimality for the real-valued objective.

The ranking replicates the paired comparison on this small family. The result is only a calibration at  $n = 8$  for the same objective. It does not bound an unknown gap at  $n = 32$  or 64, and it does not transfer to QAPLIB.

### 7.5. *Secondary boundary studies*

Four older studies delimit the claim. First, unfiltered one-step construction loses completion reliability as fill approaches 0.95; this motivated explicit completion handling. Second, feasibility experiments up to 256 facilities test packing rules rather than the proposed M1 search and are not evidence of learned or certified scalability. Third, QAPLIB removes geometric feasibility and favors swap improvement, but its optima say nothing about the gap of the layout objective. Fourth, direct transfer to integer-converted MCNC/GSRC floorplans is not competitive with topological fixed-outline methods; conversion changes rectangle geometry and dead space. Selected tables are in the Online Supplement and the complete negative record is in the archive. None of these studies is used to strengthen the main comparison.

## 8. Discussion

The comparison from the same initial layout clarifies the interpretation of the paper. A stored feasible continuation is not by itself evidence of better performance. Final verification with an incumbent has the same feasible-return coverage and the same non-worsening output property. What matters is whether intermediate completion work changes the quality-time tradeoff.

The results depend on density and cutoff. At fill 0.95, partial repair is better than final-verification construction, both spatial destroy controls and reuse without repair. The reuse contrast and event decomposition show that successful nonempty repairs matter. However, the one-second comparison with full witness regeneration does not establish a difference. At fill 0.90, complete constructions are easier and full regeneration is better. Maintaining and repairing a witness is useful near a packing barrier; outside that regime its extra structure need not improve the incumbent.

This interpretation also separates two incumbents that were mixed in the older formulation. The compatible completion is state dependent and supports the next commitment. The best complete layout is global to the run and protects output quality. Fortified rollout couples these roles through a completed-objective acceptance rule. M1 permits marginal ranking, but keeps the roles separate and verifies every full candidate before it can update the output incumbent.

The earlier penalty, mask and learning experiments remain diagnostic. A reward penalty does not enforce feasibility, and a one-step mask can strand facilities. A predictive filter with AUC 0.902 does not replace verification.



Beam and regret controls also show that learning is not a general replacement for search. These results motivate completion-aware construction, but the primary computational claim rests on the classical matched-time experiment.

The small CP-SAT and QAPLIB studies have limited roles. The former calibrates methods for the same objective at  $n = 8$ , with an explicit integer-to-real error allowance. The latter shows that swap search is strong when geometric feasibility is absent. Neither study supplies an optimality bound for the dense 32- or 64-facility cells.

## 9. Limitations

1. **The primary evidence is short-horizon.** The dense confirmation uses 50 instances per cell, two evaluation seeds and post-initialization cutoffs of 0.1, 0.3 and 1.0 seconds. It supports a short-budget claim, not long-run dominance. The boundary check has only 20 attempted instances per cell.
2. **Initialization remains incomplete.** Best-contact coverage is 0.94 to 1.00 in the new dense cells. Quality inference is therefore conditional on the common initialized set. No result establishes that the initializer covers all feasible instances.
3. **Repair controls are not exhaustive.** B2 is a documented, validation-tuned spatial destroy-and-repair search, and B2S checks destroy sizes 2 to 4. They are not a full adaptive large-neighborhood search portfolio. Different destroy rules, CP-SAT repair, or longer tuning may improve the control. M1’s cap of four and exact marginal candidate order are also empirical choices.
4. **The representation and objective limit external validity.** The core experiments use fixed-orientation axis-aligned rectangles on a lattice, without corridors or doors. Rotation is enabled only in the separate MCNC/GSRC transfer study. Integer conversion changes those benchmark geometries, and the absolute-coordinate representation is less competitive than topological fixed outline methods.
5. **Optimality evidence is local.** CP-SAT closes 45 of 60 scaled integer models at  $n = 8$ . The real-objective allowance is small but nonzero, and neither these bounds nor QAPLIB optima transfer to larger sizes or another objective.
6. **Hardware is mixed only in secondary comparisons.** B0 to B3 and M0 to M1 run on the same CPU path. Earlier learned inference

uses a GPU, so its wall time is not a hardware-normalized speedup and CPU-only policy inference is not reported.

## 10. Conclusion

This paper asks whether maintaining a completion is useful after a feasible layout is already known. Feasible return alone cannot answer this question, because any method can retain the same initial layout and verify complete candidates at the end. We therefore start every method from the same verified layout and use the same time budget.

Partial witness repair has a clear but limited advantage. At fill 0.95 and 1.0 second, it improves the common initial objective by 7.98% to 9.65% across two sizes and two geometry families. Final-verification construction improves it by 1.72% to 4.44%. The pooled gains over reuse without repair and spatial destroy and repair are 6.39 and 7.02 percentage points. No difference from full regeneration is established at this cutoff, and full regeneration is better at fill 0.90. These results support partial repair for dense packing under a short budget, but not a general advantage across facility layout problems.

The method uses the classical feasible-continuation argument of fortified rollout. Our contribution is the layout-specific repair rule, the distinction between the compatible witness and the best complete layout, and a matched-time experiment showing where partial repair helps and where it does not. Every reported incumbent passes an independent verifier. Small-instance CP-SAT claims are also stated for the scaled model with an explicit transfer bound to the real-valued objective.

## Acknowledgements

Yeoneung Kim is supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2023-00219980, RS-2026-25490291) and Yonsei University.

*Declaration of generative AI and AI-assisted technologies in the manuscript preparation process.* During the preparation of this work, the authors used OpenAI Codex and Anthropic Claude for language editing, implementation assistance, and checking of numerical consistency within the manuscript. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

## Data and code availability

A self contained reproducibility archive (`code_submission`) containing the v5 code, trained policies, benchmark instances, result files and figure/table scripts accompanies this submission as supplementary material; it includes a mapping from scripts to tables and the verification entry points. The same versioned archive is publicly available at [https://github.com/yeoneung/layout\\_optimization](https://github.com/yeoneung/layout_optimization).

## Supplementary material

An Online Supplement contains four secondary experiments referenced from the text: zero shot transfer across briefs (S1), quality and diversity fronts for all three method families (S2), amortized training cost (S3) and the discrete and continuous action granularity comparison (S4), together with the full improvement formulation ablation (S5), a qualitative certificate gallery (S6), the complete MCNC/GSRC file conversion (S7) and the large instance feasibility record (S8). The full budget and mechanism tables for the main comparison are in S9, and the diagnostic evidence removed from the focused main text is collected in S10.

## Declaration of competing interests

The authors declare no conflicts of interest.

## Appendix A. Objective, in full

*Adjusted distance.* Write  $c_i$  for facility  $i$ 's center and  $t = |\Delta y|/|\Delta x|$  for the slope of the segment joining the two centers. The directional radius  $r_i$  is the distance from  $c_i$  to the boundary of  $i$ 's rectangle along that segment: the ray exits through a vertical edge when  $t \leq h_i/w_i$ , at offsets  $(w_i/2, t w_i/2)$  from the center, and through a horizontal edge otherwise, at offsets  $(h_i/2t, h_i/2)$ ;  $r_i$  is the Euclidean norm of the offsets. The adjusted distance is  $d_{ij} = \|c_j - c_i\|_2 - r_i - r_j$ , which is zero when the rectangles touch along the segment. Its normalizer  $\bar{d}_{ij}$  is the same quantity evaluated with  $c_i$  at its lower left extreme  $(w_i/2, h_i/2)$  and  $c_j$  at its upper right extreme  $(W - w_j/2, H - h_j/2)$ , floored at  $10^{-12}$ .

*Pair and boundary terms.* With  $\nu_{ij} = \min(d_{ij}/\bar{d}_{ij}, 1)^\rho$ ,

$$c_{ij} = \begin{cases} A_{ij} \nu_{ij} & A_{ij} \geq 0 \\ -A_{ij} (1 - \nu_{ij}) & A_{ij} < 0, \end{cases}$$

zeroed if  $i$  and  $j$  overlap. The boundary term is

$$b_i = w_e h_i s_i \mathbf{1}[i \text{ flush left or right}] \\ + w_e w_i p_i \mathbf{1}[\text{flush bottom}] + w_e w_i q_i \mathbf{1}[\text{flush top}],$$

where  $(s_i, p_i, q_i) \in \{0, 1\}^3$  is the wall preference archetype of  $i$ 's type;  $b_i$  is zeroed whenever  $i$  overlaps anything.

*Constants and the terms off  $\mathcal{F}$ .* The constants are  $w_{\text{adj}} = w_e = 2$  and  $\rho = 1$ . With  $A(g)$  the total pairwise overlap area and  $P = w_{\text{ov}} A(g)$ , the full evaluator is the pair sum plus the boundary sum plus  $P + B(P)$ . Here  $w_{\text{ov}} = -8$  and

$$B(P) = \begin{cases} \beta_{\text{max}} = 400, & P = 0, \\ \beta_{\text{mid}}(1 - P/\tau)^2, & \tau \leq P < 0, \\ 0, & P < \tau, \end{cases}$$

where  $\beta_{\text{mid}} = 200$  and  $\tau = -200$ . Therefore  $P = 0$  and  $B(P) = \beta_{\text{max}}$  on  $\mathcal{F}$ , exactly as in Eq. (2); off  $\mathcal{F}$ , the overlap penalty and graded bonus apply.

*Generated suite distributions.* Each instance draws its mean facility area from  $U(9, 30)$ , capped at 0.95 of the budget for each facility,  $44^2 \cdot \text{fill}/n$ ; a plate aspect ratio from  $U(1, 1.8)$ ; plate sides meeting the implied area, each at most 44; a guillotine partition with minimum side 2, shrunk to the target fill; a type count  $T$  uniform on  $\{6, \dots, \min(12, n)\}$ ; a symmetric integer affinity table on types, entries uniform on  $\{-8, \dots, 8\}$  and zeroed with probability 0.25; and a wall preference archetype per type. Three objective constants are drawn again once per suite cell, spanning the range the two reference scenarios occupy:  $w_{\text{ov}} \sim U(-12, -6)$ ,  $\beta_{\text{max}} \sim U(320, 520)$  and  $\tau \sim U(-280, -160)$ ; all other constants are fixed at the values above.

## Appendix B. Policy and training

Facility tokens are embedded from features for each facility (size, placed flag, position, boundary coefficients, and the affinity and distance rows

Table B.5: Training runs behind every learned result, from the shipped checkpoint histories and predictive filter metadata. Budgets end at an iteration boundary, so nominal 1.2M budgets record as 1,196,032 or 1,198,080.

| checkpoint         | role                         | budget    |            | min                         | deviations              |
|--------------------|------------------------------|-----------|------------|-----------------------------|-------------------------|
| S_dord_seed1/2     | density, $n = 32$            | 1 196 032 |            | 34/50                       |                         |
| S64_seed1/2        | density, $n = 64$            | 1 196 032 |            | 34/34                       |                         |
| S_bench_seed1/2/3  | main suite                   | 1 196 032 | 45/44/42   |                             | random order            |
| D1_constr_random   | transfer (S1)                | 1 198 080 | 17         | 64 env, 2 ep, mb 256, 48 ch |                         |
| B1_constr_fixed    | OFFICE                       | 1 998 848 | 34         | 2M steps                    |                         |
| B2_constr_hospital | CLINIC                       | 899 072   | 7          | 0.9M steps, 64 env          |                         |
| A0/A0'/A1/A2/C1    | improvement ablation         | 688 128   | 3.2 to 3.4 |                             | five separate runs      |
| C1'                | continuous head (S4)         | 688 128   | 2.5        |                             | continuous displacement |
| viability_net      | predictive filter diagnostic | 4 000 ep. | 0.28       |                             | grouped validation      |

against all facilities) and mixed by self attention; a pointer head selects the facility. Position selection is a fully convolutional score over the plate from dilated residual convolutions on seven planes (occupancy, the legality mask, the affinity of the facility being placed toward whichever facility owns each cell, the boundary bonus it would earn, a mask for positions within bounds and two coordinate planes), modulated channelwise by the selected facility’s embedding. The encoder has no separate embedding for each facility index, which permits transfer across briefs at fixed  $n$ . The reward is the exact marginal of Proposition 1, so the episode return is the objective of the completed layout up to a constant.

Unless a row of Table B.5 says otherwise, every constructive policy shares one PPO configuration:  $\gamma = 1$ , GAE  $\lambda = 0.95$ , clip 0.2, Adam at  $3 \times 10^{-4}$ , 3 epochs per batch, minibatch 512, value coefficient 0.5, gradient norm clip 0.5, entropy coefficient 0.01 decayed linearly to zero at 80% of training with a floor of 5% of its initial value, token width 128, three attention blocks, 64 position head channels and 128 parallel environments. For every policy, the final checkpoint is used; no checkpoint selection based on validation is performed. The predictive filter is a separate supervised diagnostic: it retains the lowest validation loss state over 4 000 epochs and fixes its seven thresholds on that same grouped validation set before any test evaluation. The five improvement MDP checkpoints of Section 4 (tabulated in the Online Supplement, S5) are trained separately at a matched budget of 688,128 steps (193 to 203s each). Wall clock for policy training is on one RTX 4080 SUPER; the predictive filter was trained on CPU in 17s.

## References

- Adya, S.N., Markov, I.L., 2003. Fixed-outline floorplanning: enabling hierarchical design. *IEEE Transactions on Very Large Scale Integration Systems* 11, 1120–1135. doi:10.1109/TVLSI.2003.817546.
- Ahn, J., Lee, K., Lee, K., Kim, Y., 2026. RoomLayoutRL: Legacy scenario implementation. GitHub repository. URL: <https://github.com/KyunghyunLee/RoomLayoutRL>. development repository; accessed 29 August 2026.
- Gomes de Alvarenga, A., Negreiros-Gomes, F.J., Mestria, M., 2000. Meta-heuristic methods for a class of the facility layout problem. *Journal of Intelligent Manufacturing* 11, 421–430.
- Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2017. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940* .
- Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research* 290, 405–421.
- Bertsekas, D.P., 2005. Rollout Algorithms for Constrained Dynamic Programming. Technical Report LIDS Report 2646. Massachusetts Institute of Technology, Laboratory for Information and Decision Systems. URL: [https://www.mit.edu/~dimitrib/Rollout\\_Constrained.pdf](https://www.mit.edu/~dimitrib/Rollout_Constrained.pdf).
- Bertsekas, D.P., 2020. Constrained multiagent rollout and multidimensional assignment with the auction algorithm. *arXiv preprint arXiv:2002.07407* doi:10.48550/arXiv.2002.07407.
- Burkard, R.E., Karisch, S.E., Rendl, F., 1997. QAPLIB – a quadratic assignment problem library. *Journal of Global Optimization* 10, 391–403.
- Chang, Y.C., Chang, Y.W., Wu, G.M., Wu, S.W., 2000. B\*-Trees: A new representation for non-slicing floorplans, in: *Proceedings of the 37th Design Automation Conference*, pp. 458–463. doi:10.1109/DAC.2000.855354.
- Drira, A., Pierreval, H., Hajri-Gabouj, S., 2007. Facility layout problems: A survey. *Annual reviews in control* 31, 255–267.

- Fowler, R.J., Paterson, M.S., Tanimoto, S.L., 1981. Optimal packing and covering in the plane are NP-complete. *Information Processing Letters* 12, 133–137. doi:10.1016/0020-0190(81)90111-3.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A., 2019. Exact combinatorial optimization with graph convolutional neural networks, in: *Advances in Neural Information Processing Systems*, pp. 15554–15566.
- Glover, F., 1989. Tabu search—part i. *ORSA Journal on Computing* 1, 190–206.
- Google OR-Tools, 2026. CP-SAT solver. Official documentation. URL: [https://developers.google.com/optimization/cp/cp\\_solver](https://developers.google.com/optimization/cp/cp_solver). accessed 6 September 2026.
- Haralick, R.M., Elliott, G.L., 1980. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence* 14, 263–313.
- Heinbach, B., Burggräf, P., Wagner, J., 2023. Deep reinforcement learning for layout planning—an MDP-based approach for the facility layout problem. *Manufacturing Letters* 38, 40–43.
- Ikedo, H., Nakagawa, H., Tsuchiya, T., 2022. Towards automatic facility layout design using reinforcement learning., in: *FedCSIS (Communication Papers)*, pp. 11–20.
- Kakooee, R., Dillenburger, B., 2024. Reimagining space layout design through deep reinforcement learning. *Journal of Computational Design and Engineering* 11, 43–55.
- Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. *Science* 220, 671–680.
- Kool, W., van Hoof, H., Welling, M., 2019. Attention, learn to solve routing problems! *International Conference on Learning Representations* .
- Koopmans, T.C., Beckmann, M., 1957. Assignment problems and the location of economic activities. *Econometrica* 25, 53–76.
- Luo, G., Zhou, X., Feng, L., Liu, J., Liu, P., Liao, Y., Shan, W., Qi, H., 2025. Controllable and flexible residential floor plan layout design based on

- multi-agent deep reinforcement learning with layout prior size and similar experience abandon. *Advanced Engineering Informatics* 68, 103702.
- Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E., 2021. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research* 134, 105400.
- Mirhoseini, A., Goldie, A., Yazgan, M., et al., 2021. A graph placement methodology for fast chip design. *Nature* 594, 207–212.
- Murata, H., Fujiyoshi, K., Nakatake, S., Kajitani, Y., 1996. VLSI module placement based on rectangle-packing by the sequence-pair. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 15, 1518–1524. doi:10.1109/43.552084.
- Ow, P.S., Morton, T.E., 1988. Filtered beam search in scheduling. *International Journal of Production Research* 26, 35–62.
- Pérez-Gosende, P., Mula, J., Díaz-Madroño, M., 2021. Facility layout planning. an extended literature review. *International Journal of Production Research* 59, 3777–3816.
- Potvin, J.Y., Rousseau, J.M., 1993. A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. *European Journal of Operational Research* 66, 331–340.
- Røpke, S., Pisinger, D., 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science* 40, 455–472. doi:10.1287/trsc.1050.0135.
- Şahin, R., Niroomand, S., Durmaz, E.D., Molla-Alizadeh-Zavardehi, S., 2020. Mathematical formulation and hybrid meta-heuristic solution approaches for dynamic single row facility layout problem. *Annals of Operations Research* 295, 313–336.
- Schulman, J., et al., 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* .
- Singh, S.P., Sharma, R.R., 2006. A review of different approaches to the facility layout problems. *The International Journal of Advanced Manufacturing Technology* 30, 425–433.



- Taillard, É., 1991. Robust taboo search for the quadratic assignment problem. *Parallel Computing* 17, 443–455.
- Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A., 2021. Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems* 33, 5057–5069.